

Otulia Web: The Complete Developer Guide

Table of Contents

- [1. Chapter 1: Introduction & Architecture](#)
 - [2. Chapter 2: The Data Layer \(MongoDB & Models\)](#)
 - [3. Chapter 3: Backend Services & API \(Node.js & Express\)](#)
 - [4. Chapter 4: Authentication & Authorization](#)
 - [5. Chapter 5: Frontend Architecture \(React & Vite\)](#)
 - [6. Chapter 6: Asset Management & Cloudinary](#)
-

Chapter 1: Introduction & Architecture

1.1 Project Overview

Otulia Web is a luxury asset marketplace designed to facilitate the buying, selling, and renting of high-end assets such as Cars, Yachts, Estates, and Bikes. The platform supports multiple user roles, including standard users, agents/dealers, and administrators.

1.2 Technology Stack

The application is built using the MERN stack (MongoDB, Express, React, Node.js) with modern tooling:

- **Frontend:** React 19, Vite, React Router DOM, Tailwind CSS (v4), Context API / Redux for state management, and Google OAuth for authentication.
- **Backend:** Node.js, Express.js.
- **Database:** MongoDB with Mongoose ODM.
- **Storage:** Cloudinary for managing images and verification documents.
- **Authentication:** JWT (JSON Web Tokens) and Google Auth Library.
- **Payments:** Stripe and PayPal integrations (configured in package.json).

1.3 System Architecture

The repository is structured as a monorepo containing two main directories:

- `/client` : The React frontend application.
- `/server` : The Express backend application.

The client acts as a Single Page Application (SPA) communicating with the server via RESTful APIs mapped under the `/api/*` endpoint. The server handles business logic, database transactions, and file interactions (via Cloudinary).

Chapter 2: The Data Layer (MongoDB & Models)

The core entities of the application are defined in `server/models`. The data layer heavily utilizes Mongoose schemas to enforce structure and validation.

2.1 The User Model (`User.model.js`)

The User model is central to the application, managing authentication, roles, and user-specific data.

- **Roles:** Defined via an enum: `['user', 'agent', 'admin']`.
- **Plans:** Supports tiered access via `plan: ['Freemium', 'Premium Basic', 'Business VIP']`.
- **Histories:** Tracks `favorites`, `myListings`, `boughtHistory`, `rentedHistory`, and `soldHistory`.

- **Verification:** Contains specific fields (`isVerified` , `verificationStatus` , `verificationDocuments`) to handle agent/dealer KYC flows.

2.2 Asset Models (`CarAsset.model.js` , `Listing.model.js` , etc.)

The system separates generic user listings from highly detailed luxury assets.

- **Listing.model.js:** A generic model for standard user listings, capturing basic info (title, images, price, category, location, agent info).
 - **Domain-Specific Assets:** Models like `CarAsset.model.js` define extremely granular schemas. For example, a `CarAsset` tracks `power` , `topSpeed` , `cylinderCapacity` , `matchingNumbers` , and `accidentHistory` .
 - **Polymorphic Relationships:** User favorites and histories use dynamic referencing (`refPath`) to point to either a `Listing` , `CarAsset` , `EstateAsset` , `YachtAsset` , or `BikeAsset` .
-

Chapter 3: Backend Services & API (Node.js & Express)

3.1 Server Initialization (`server/index.js`)

The server entry point connects to MongoDB, sets up global middleware (CORS, JSON parsing), and registers all route modularly.

- Route prefixes are clean, such as `/api/auth` , `/api/assets` , `/api/listings` , `/api/admin` .
- The backend is configured to serve the built Vite frontend (`../client/dist`) in production or fallback scenarios, creating a unified deployment if needed.

3.2 Key API Routes

- `auth.routes.js` : Handles standard email/password registration, login, Google OAuth, and profile updates. It also handles document uploads for verification.
 - `assets.routes.js` / `listing.routes.js` : Handle CRUD operations for assets.
 - `lead.routes.js` : Manages inquiries made on specific assets, connecting buyers with agents.
-

Chapter 4: Authentication & Authorization

4.1 JWT & Middleware

Authentication relies on stateless JSON Web Tokens.

- When a user logs in or registers, the server signs a JWT containing the user's `id` , `role` , and `email` , valid for 7 days.
- `auth.middleware.js` (inferred): Protects secure routes by verifying the Bearer token in the `Authorization` header. If valid, it appends the decoded user payload to `req.user` .

4.2 Google OAuth

The application utilizes `@react-oauth/google` on the frontend and `google-auth-library` on the backend.

- The frontend requests an ID token from Google.
 - The backend verifies this token securely with Google's servers.
 - If the user exists, their account is returned. If not, an account is automatically provisioned and optionally linked to existing email-based accounts.
-

Chapter 5: Frontend Architecture (React & Vite)

5.1 Entry Point (`client/src/main.jsx` & `App.jsx`)

- The application uses `BrowserRouter` for client-side routing.
- High-level providers wrap the app: `<GoogleOAuthProvider>` , `<AuthProvider>` , and `<CartProvider>` .
- `App.jsx` maintains a mapping of routes to page components (`Home` , `Shop` , `Asset` , `AdminDashboard` , etc.). It dynamically hides the `Footer` on specific pages (like login/signup) to maintain aesthetic focus.

5.2 Context Providers (`AuthContext.jsx`)

State that spans the entire application is managed via Contexts.

- `AuthContext` provides a seamless abstraction over the API. It manages local storage for the JWT, fetches the current user profile on load (`/api/auth/me`), and exposes methods like `login` , `signup` , `googleLogin` , and `logout` .

5.3 User Experience (UX)

The frontend utilizes a custom `SplashScreen` component on load and implements scroll-to-top logic on route transitions to ensure a premium feel consistent with a luxury marketplace.

Chapter 6: Asset Management & Cloudinary

6.1 Media Storage

Instead of storing images on the server's disk, the application deeply integrates with Cloudinary.

- `multer` and `multer-storage-cloudinary` are used in routes to seamlessly upload images directly from the request stream to Cloudinary folders.
- Folders are dynamically generated per user/agent (e.g., `getFolderPaths(req.user.email)`), ensuring organized media assets.

6.2 Data Synchronization

When an agent updates their profile (e.g., uploading a new company logo or profile picture), the backend utilizes a utility function (`updateUserAssetsAgent`) to propagate these updates to all assets owned by that agent. This denormalization strategy ensures that asset queries are fast and do not require heavy database `JOIN` / `populate` operations for agent details.